

UCS503 - Software Engineering

VeriHire: AI-Powered Job Posting Verification System

An Explainable Machine Learning System for Detecting Suspicious Job and Internship Postings

Submitted by:

Nidhish	1024160013
Geet Rawal	1024160012
Parvesh	1024160017

Faculty Advisor:

Dr. Jeelani

Progress Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

September 15, 2026

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	3
2	Methodology and Design	4
2.1	System Architecture	4
2.1.1	High-Level Design	4
2.1.2	Technical Stack	4
2.2	Machine Learning Methodology	5
2.2.1	Data and Feature Strategy	5
2.2.2	Model Selection Strategy	6
2.2.3	Risk Assessment and Explainability	6
2.3	Implementation Progress	6
2.3.1	Module 1: Project Definition and Software Engineering Artifacts .	6
2.3.2	Module 2: Authentication and Application Foundation	7
2.3.3	Module 3: Frontend Product Interface	7
2.3.4	Module 4: Verification Dashboard	7
2.3.5	Module 5: Verification Result Design	8
3	Current Status and Evaluation	9
3.1	Development Status	9
3.2	Testing Strategy	9
3.3	Current Challenges	10
4	Conclusion	11
4.1	Summary of Progress	11
4.2	Key Findings	11
4.3	Future Work and Recommendations	12
4.4	Final Remarks	12

Chapter 1

Introduction

1.1 Background and Context

Online job and internship platforms have significantly simplified the process of discovering employment opportunities. However, the increasing number of fraudulent and misleading postings has created a serious risk for students and job seekers. Fraudulent postings may use unrealistic compensation, registration or application fees, generic contact information, urgency-based language, or incomplete organizational details to manipulate applicants.

VeriHire is being developed as an AI-powered verification platform that analyzes job and internship postings and identifies suspicious characteristics before a user applies. Rather than treating the problem as a simple binary “real or fake” classification, the system is designed to provide a risk assessment supported by interpretable suspicious signals.

The project combines a web-based verification interface with a machine learning pipeline so that users can submit a posting, receive a risk assessment, inspect the detected signals, and review their previous verification activity.

1.1.1 Problem Statement

Existing job platforms primarily focus on discovering and applying to opportunities and generally do not provide a dedicated mechanism for analyzing whether a posting contains characteristics commonly associated with fraudulent recruitment.

Students and inexperienced applicants may therefore have difficulty identifying suspicious postings, especially when scams are presented using professional-looking job descriptions. Manually evaluating every posting is time-consuming and subjective.

VeriHire addresses this gap by designing an explainable verification system that analyzes textual and contextual features of job and internship postings, assigns a risk level, provides a confidence estimate, and highlights the signals contributing to the assessment.

1.2 Project Objectives

The current project objectives are:

1. **AI-Based Risk Assessment:** Develop a machine learning pipeline capable of estimating the risk associated with a job or internship posting.
2. **Suspicious Signal Detection:** Identify interpretable indicators such as payment requests, unrealistic compensation, suspicious contact information, urgency-based language, and incomplete organizational information.
3. **Explainable Results:** Present the risk assessment together with the signals that contributed to the result rather than returning only a binary prediction.
4. **Web-Based Verification:** Provide a React-based interface through which users can submit job posting text or supported URLs for analysis.
5. **Verification History:** Maintain a history of previously analyzed postings so users can revisit earlier results.
6. **Analytics Dashboard:** Provide an authenticated dashboard containing verification statistics, risk distribution, activity trends, and frequently detected suspicious signals.
7. **Professional Product Experience:** Develop a consistent dark, premium interface connecting the interactive constellation-based landing experience with the authenticated verification environment.

Chapter 2

Methodology and Design

2.1 System Architecture

VeriHire follows a modular web application architecture separating the presentation layer, verification logic, machine learning pipeline, and persistent application services.

2.1.1 High-Level Design

The current architecture is organized into the following major layers:

1. **Presentation Layer:** A React-based frontend providing the landing page, posting checker, verification result interface, authentication screens, and dashboard.
2. **Verification Layer:** Processes submitted job or internship posting content and prepares it for analysis.
3. **Machine Learning Layer:** Extracts relevant textual and contextual features and produces a fraud-risk assessment.
4. **Explainability Layer:** Converts model output and rule-based indicators into understandable suspicious signals that can be presented to the user.
5. **Application Data Layer:** Stores authentication, verification history, and other application-level information required by the product.

2.1.2 Technical Stack

The current project uses the following technologies and design choices:

- **Frontend Framework:** React with Vite.
- **Language:** JavaScript / JSX.

- **Styling:** Tailwind CSS and custom design tokens where applicable.
- **Icons:** Lucide React.
- **Backend:** REST-based backend services for authentication, verification, and application data.
- **Machine Learning:** Python-based machine learning pipeline.
- **Data Processing:** Pandas and NumPy.
- **Machine Learning Libraries:** scikit-learn and related model tooling.
- **Database:** Application database layer for user and verification data.
- **Development Environment:** Git/GitHub with Antigravity-assisted development workflow.

2.2 Machine Learning Methodology

The ML component is being designed as a risk-assessment system rather than a black-box binary classifier.

2.2.1 Data and Feature Strategy

The training and validation strategy is based on labeled fraudulent-job data together with real-world job posting examples and manually engineered patterns relevant to Indian recruitment scams.

Important feature groups include:

- Payment or registration fee indicators.
- Salary and stipend realism.
- Suspicious or generic contact information.
- Email and domain characteristics.
- Job-description language and suspicious phrases.
- Urgency and pressure-based wording.
- Experience and qualification inconsistencies.
- Completeness of company and job information.
- Other textual and contextual indicators associated with suspicious recruitment.

2.2.2 Model Selection Strategy

Candidate supervised classification models are being evaluated using standard classification metrics such as precision, recall, and F1-score.

The current design considers tree-based ensemble learning, particularly Random Forest and gradient-boosting approaches, as strong candidates because they can model non-linear relationships between heterogeneous risk features.

The final production model will be selected after comparative evaluation on the prepared validation data. The model output will subsequently be transformed into a normalized risk score used by the application.

2.2.3 Risk Assessment and Explainability

The final user-facing result is intended to contain:

- **Risk Level:** LOW, MEDIUM, or HIGH.
- **Risk Score:** A normalized score representing the estimated level of concern.
- **Model Confidence:** Confidence associated with the prediction.
- **Detected Signals:** Human-readable reasons contributing to the assessment.
- **Recommendation:** A practical action for the applicant based on the detected risk.

The system is designed to communicate that a high-risk assessment is an indication for further verification and caution, rather than definitive proof that a posting is fraudulent.

2.3 Implementation Progress

2.3.1 Module 1: Project Definition and Software Engineering Artifacts

The initial project scope and system requirements have been established. The use-case and data-flow representations were reviewed and corrected to better match the actual VeriHire workflow.

The main workflow currently follows:

User → Posting Input → Analysis → Risk Assessment → Explainable Result (2.1)

2.3.2 Module 2: Authentication and Application Foundation

The basic frontend and backend foundation for user authentication has been implemented. The project includes the initial login and signup flow and the authenticated application structure required to support dashboard access.

The authentication layer is being integrated with the rest of the application so that verification activity can be associated with a user.

2.3.3 Module 3: Frontend Product Interface

A React-based interface has been developed and the major product screens are being organized into a coherent application flow.

The landing experience uses an interactive constellation/network visual as the primary brand element. The constellation represents signals and connections and establishes the technical identity of VeriHire.

The authenticated dashboard has also been implemented with the following current sections:

- Overview
- Check Posting
- History
- Analytics
- Settings
- Help / Methodology

The current dashboard contains verification KPIs, recent verification records, risk distribution, verification activity, action-required items, frequently detected signals, and system status information.

2.3.4 Module 4: Verification Dashboard

The dashboard is designed as a verification control center rather than a generic administrative interface.

The current design includes:

- Total verification count.
- High-, medium-, and low-risk posting counts.

- Recent verification table.
- Risk distribution visualization.
- Verification activity trend.
- Action-required verification items.
- Frequently detected suspicious signals.
- Model and analysis engine status.

The interface is currently being refined using a dark, near-black theme with layered surfaces, restrained green/cyan accents, and modern typography. Technical monospace typography is being limited to system metadata rather than the entire interface.

2.3.5 Module 5: Verification Result Design

The verification result is being designed as the main analytical output of the system.

The planned result structure includes:

- Overall risk classification.
- Numerical risk score.
- Model confidence.
- Detected suspicious signals.
- Evidence from the submitted posting.
- Explanation of why each signal is relevant.
- Recommended next action.

This design ensures that the ML prediction is presented together with interpretable evidence.

Chapter 3

Current Status and Evaluation

3.1 Development Status

The project has progressed from initial requirements and software engineering documentation into active implementation of the product interface, authentication foundation, dashboard, and machine learning architecture.

Component	Current Status
Project scope and requirements	Completed
Use-case and DFD refinement	Completed
React application foundation	Completed
Authentication foundation	Implemented
Landing page concept	Implemented / Refining
Dashboard overview	Implemented / Refining
Verification checker	In development
Verification result interface	In development
History interface	Planned / In development
Analytics interface	In development
ML data preparation	In progress
Model comparison and validation	In progress
Backend-frontend integration	In progress

Table 3.1: Current VeriHire Development Status

3.2 Testing Strategy

The project will evaluate the system at multiple levels:

- **Unit Testing:** Validate individual preprocessing, feature extraction, risk calculation, and utility functions.
- **Model Evaluation:** Compare candidate classifiers using precision, recall, F1-score, confusion matrix, and related classification metrics.
- **Integration Testing:** Verify the complete flow from posting submission through backend processing and result delivery.
- **Frontend Testing:** Verify authentication, navigation, posting submission, result rendering, history access, and dashboard interactions.
- **Edge-Case Testing:** Test incomplete postings, extremely short inputs, suspicious URLs, missing fields, and malformed submissions.

3.3 Current Challenges

The major challenges being addressed during the current development phase include:

1. **Explainability:** Ensuring that the ML prediction can be communicated through understandable suspicious signals.
2. **Dataset Generalization:** Ensuring that the model performs well on real-world job postings rather than only on the training dataset.
3. **Risk Calibration:** Designing meaningful LOW, MEDIUM, and HIGH risk thresholds without presenting the prediction as absolute proof of fraud.
4. **Frontend Consistency:** Maintaining a coherent visual identity between the constellation landing page and the authenticated dashboard.
5. **Backend Integration:** Connecting the implemented frontend screens with real verification and ML services while keeping the application architecture modular.

Chapter 4

Conclusion

4.1 Summary of Progress

During the current development phases, VeriHire has progressed from project definition and software engineering documentation to implementation of the core application foundation.

The authentication foundation, React frontend structure, constellation-based landing experience, and dark verification dashboard have been developed. The ML architecture has also been defined around explainable risk assessment, suspicious signal detection, and comparative evaluation of candidate classification models.

The current work is focused on completing the posting-analysis workflow, integrating the ML pipeline with the backend, implementing the detailed verification result interface, and connecting the remaining dashboard functionality with real application data.

4.2 Key Findings

The development process has established several important design decisions:

- **Risk Assessment over Binary Classification:** VeriHire should communicate levels of risk and supporting evidence instead of presenting fraud detection as an infallible binary decision.
- **Explainability is Central:** The usefulness of the system depends not only on prediction accuracy but also on explaining the suspicious signals detected in a posting.
- **Product Trust Requires Visual Consistency:** The constellation-based landing page and the authenticated dashboard need to share a consistent technical identity while serving different purposes.

4.3 Future Work and Recommendations

The next development phases will focus on:

1. Completing the ML preprocessing, feature engineering, model comparison, and validation pipeline.
2. Finalizing the production model based on validation performance.
3. Integrating the ML inference service with the backend API.
4. Completing the Check Posting workflow for text and supported URL inputs.
5. Implementing the detailed Verification Result page.
6. Completing History and Analytics functionality.
7. Connecting dashboard statistics to real verification records.
8. Adding comprehensive frontend, backend, and ML testing.
9. Preparing the system for deployment and final evaluation.

4.4 Final Remarks

VeriHire is being developed as an explainable AI-assisted verification platform for students and job seekers. The current implementation establishes the product foundation, while the remaining work will concentrate on validating the machine learning pipeline and integrating it into a reliable end-to-end verification workflow.

Bibliography

- [1] Scikit-learn Documentation,
Machine Learning in Python,
<https://scikit-learn.org/>
- [2] Pandas Documentation,
Python Data Analysis Library,
<https://pandas.pydata.org/>
- [3] React Documentation,
React: The library for web and native user interfaces,
<https://react.dev/>